



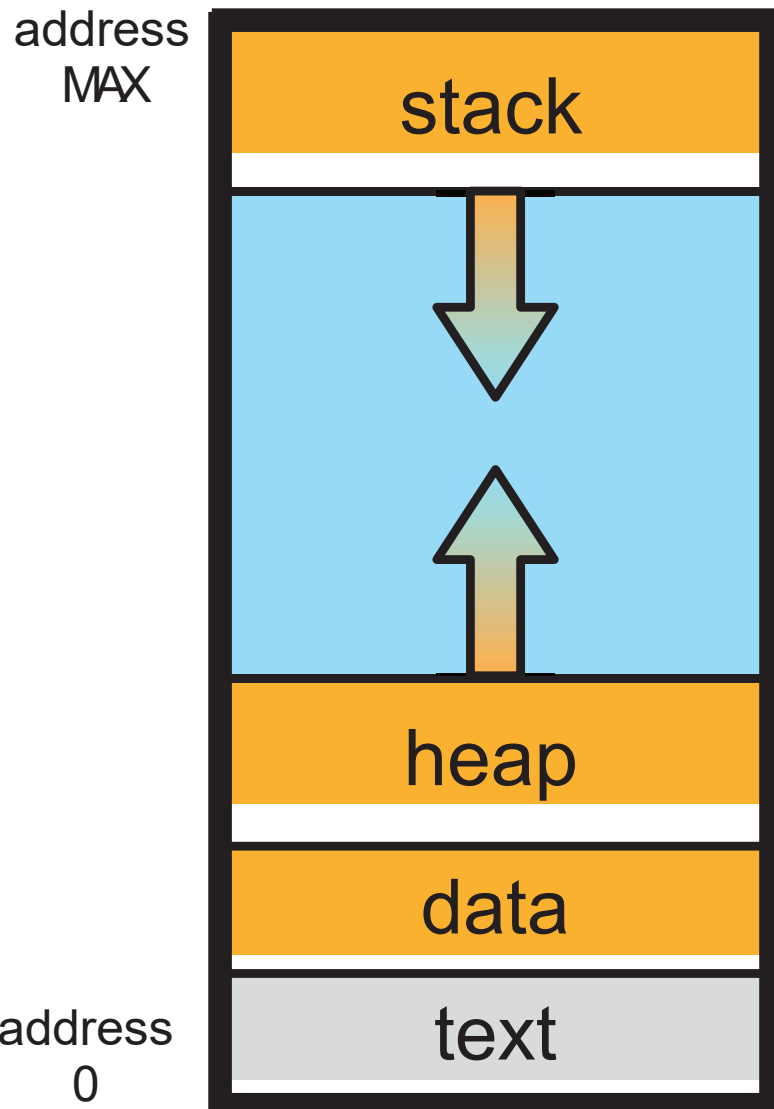
OS LAB

Process Management

Program vs Process

- **Program:** A set of instructions which is in human readable format.
A passive entity stored on secondary storage (external storage). Eg.
The code you wrote in C
- When you *run* a program, it becomes a **process**. So a process is a **program** loaded into **memory** and executing or waiting.
- A process typically executes for only a short time before it either finishes or needs to perform I/O (waiting).
- A process is an active entity and **needs resources** such as **CPU time, memory** etc.. to execute

A Process In Memory



Stack: Temporary data such as function parameters, local variables and return addresses.

The stack grows from high addresses towards lower address.

Heap: Dynamically allocated (malloc) by the program during runtime.

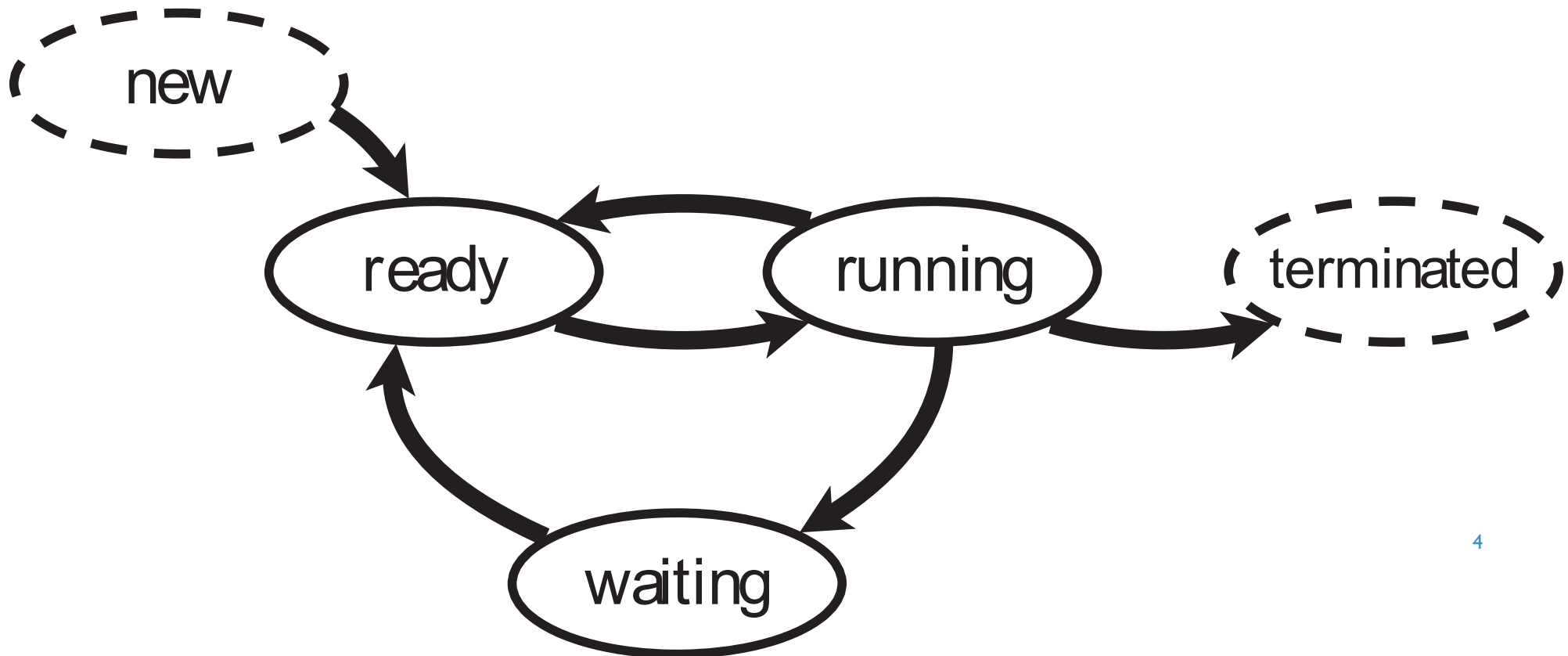
The heap grows from low addresses towards higher addresses.

Data: Statically (known at compile time) global variables and data structures.

Text: The program executable machine instructions (actual code you wrote)

Process State Transitions

In general, a process can be in one of **five** states: new, ready, running, waiting or terminated. A process transitions between the states according to the following diagram.



Example Program

example.c

```
// Example C program

#include <stdlib.h>

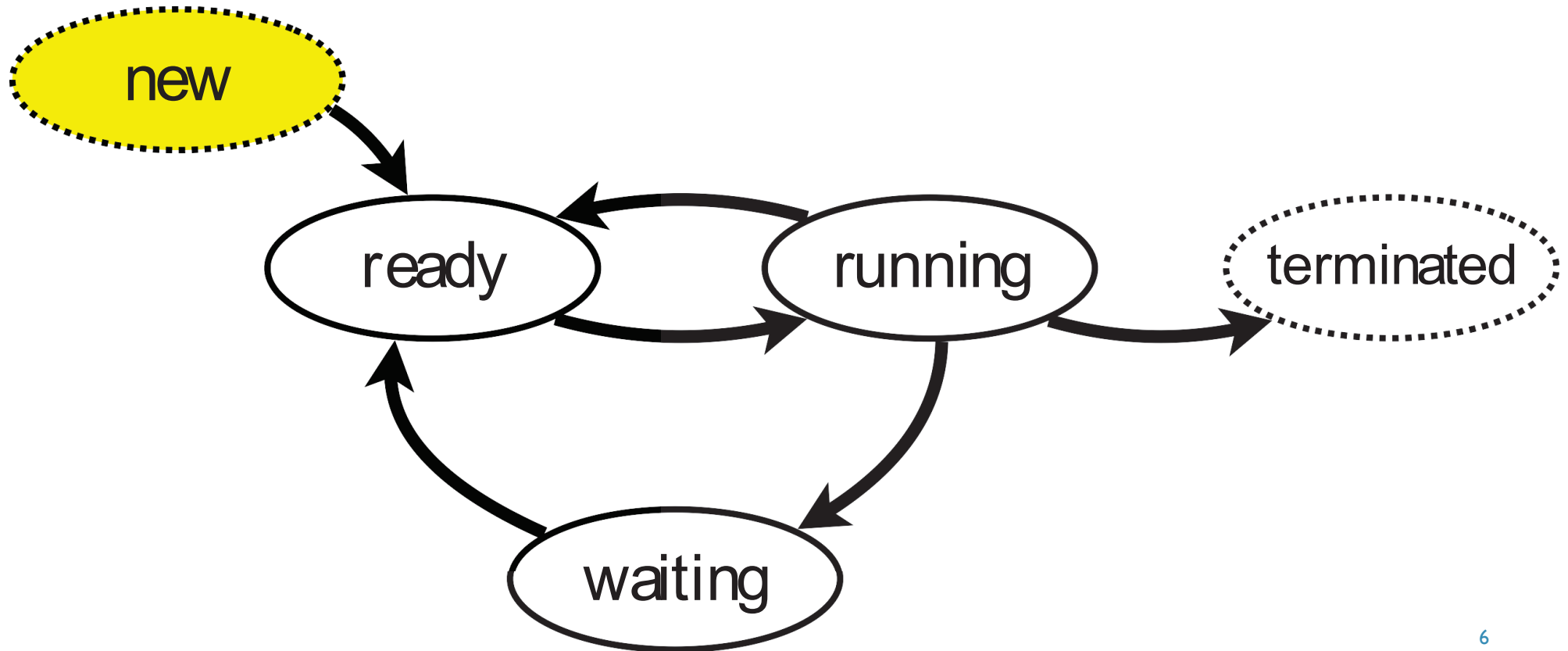
int x;

int main(void) {
    int y;
    char* str;
    str = malloc(50);
    exit(EXIT_SUCCESS);
}
```

- When we type
`gcc example.c -o example`

An executable named “example” will be created.
- When we type `./example` to run the program, the operating system must first create a process.
- To create a process, the operating system must allocate memory for the process.

To run a program, the operating system must first allocate memory for the process.



Static Memory Allocation

```
// Example C program
#include <stdlib.h>

int x;

int main(void) {
    int y;
    char* str;
    str = malloc(50);
    exit(EXIT_SUCCESS);
}
```

address MAX

memory

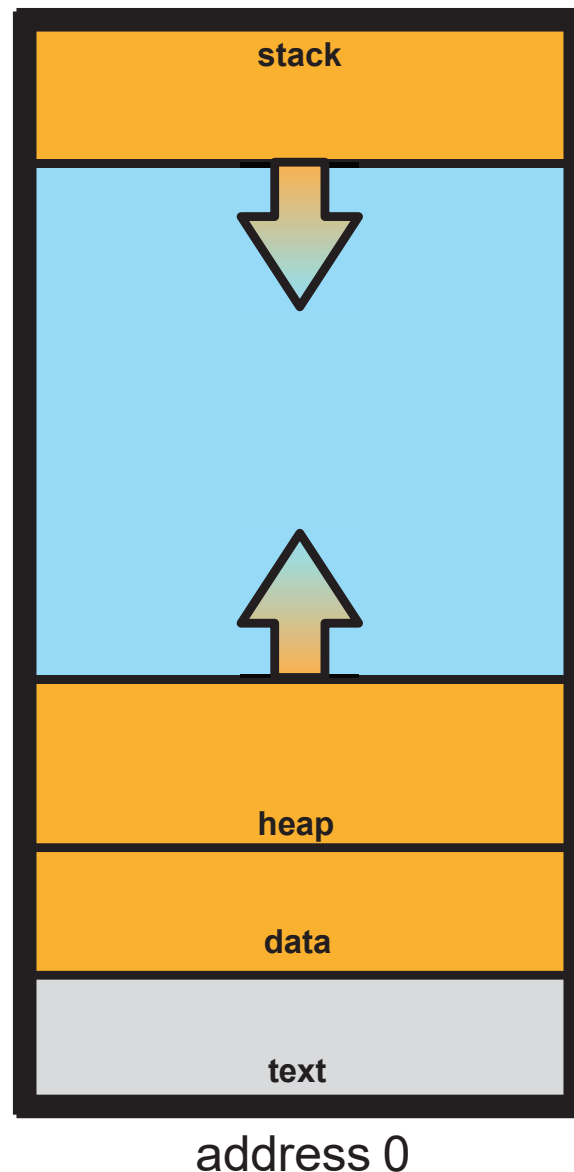
address 0

The operating system allocates a blob of memory for the new process.

```
// Example C program
#include <stdlib.h>

int x;

int main(void) {
    int y;
    char* str;
    str = malloc(50);
    exit(EXIT_SUCCESS);
}
```



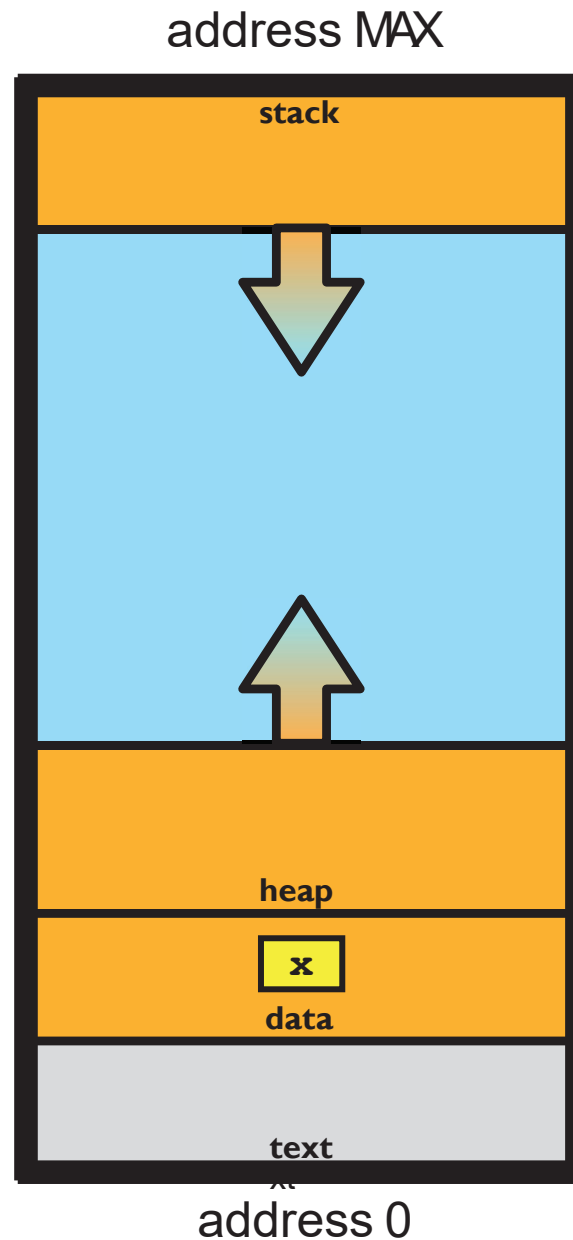
The allocated memory is divided into the following segments:

- text
- data
- heap
- stack


```
// Example C program
#include <stdlib.h>

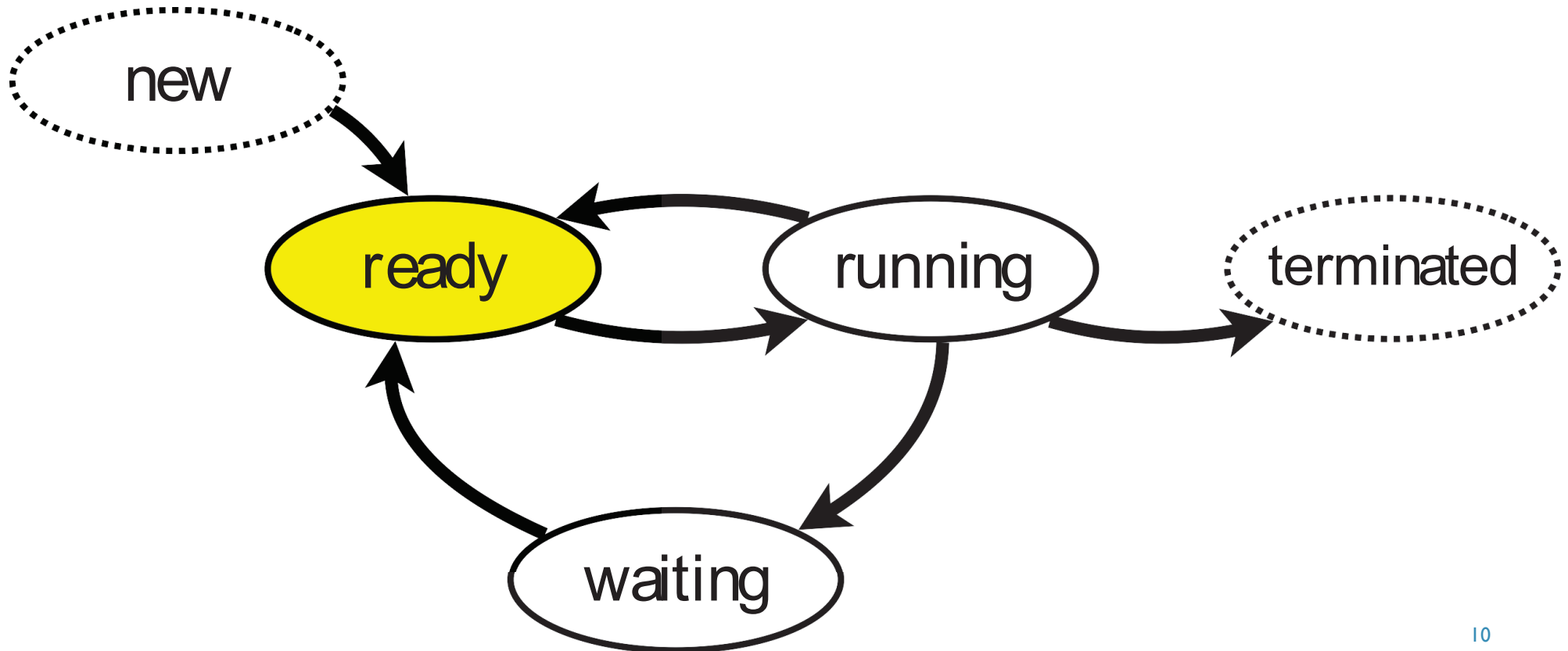
int x;

int main(void) {
    int y;
    char* str;
    str = malloc(50);
    exit(EXIT_SUCCESS);
}
```

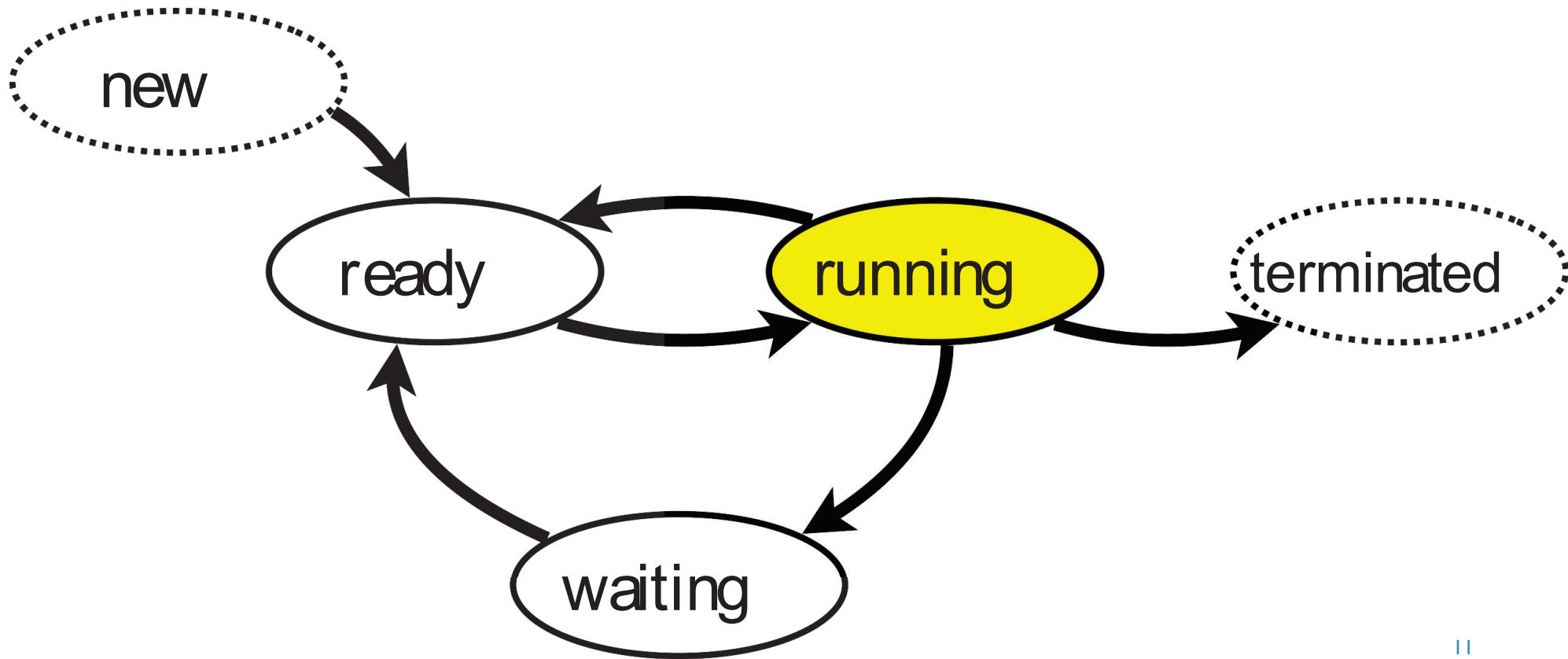


The size of the needed storage for the **global variable x** is known at compile time and storage for x is allocated in the static **data segment**.

After the operating system has allocated memory for the process, it is ready to execute and changes state from new to ready.



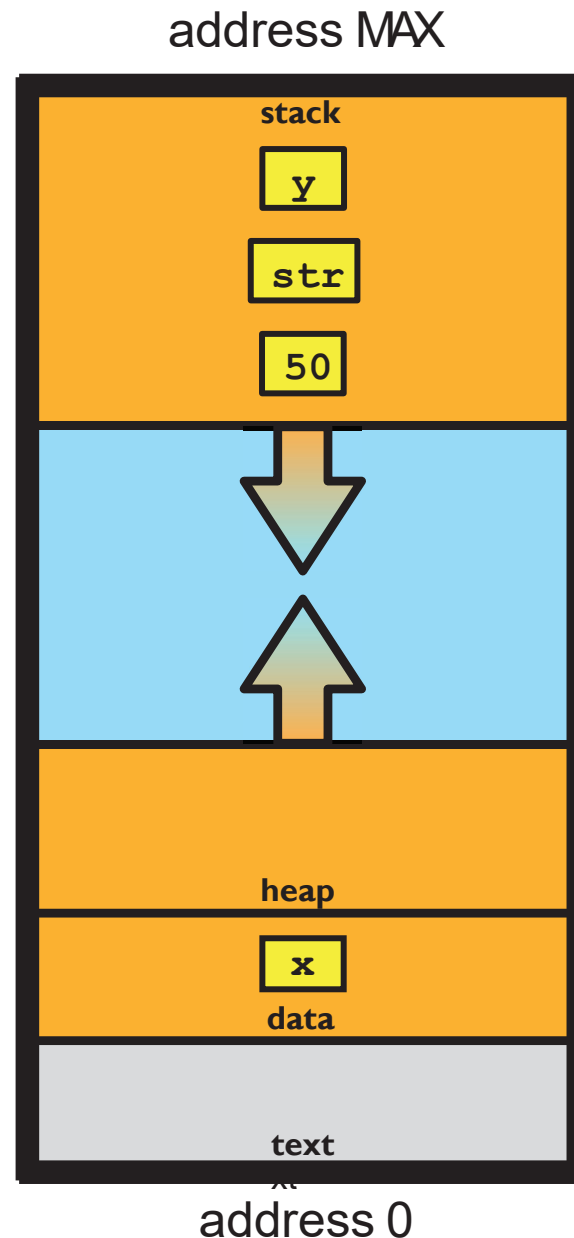
When the process is selected to execute, it changes state from ready to running.



```
// Example C program
#include <stdlib.h>

int x;

int main(void) {
    int y;
    char* str;
    str = malloc(50);
    exit(EXIT_SUCCESS);
}
```



The size of the needed storage for the **local variables** **y** and **str** are also known at compile time but these are only needed within main and storage is allocated on the **stack**.

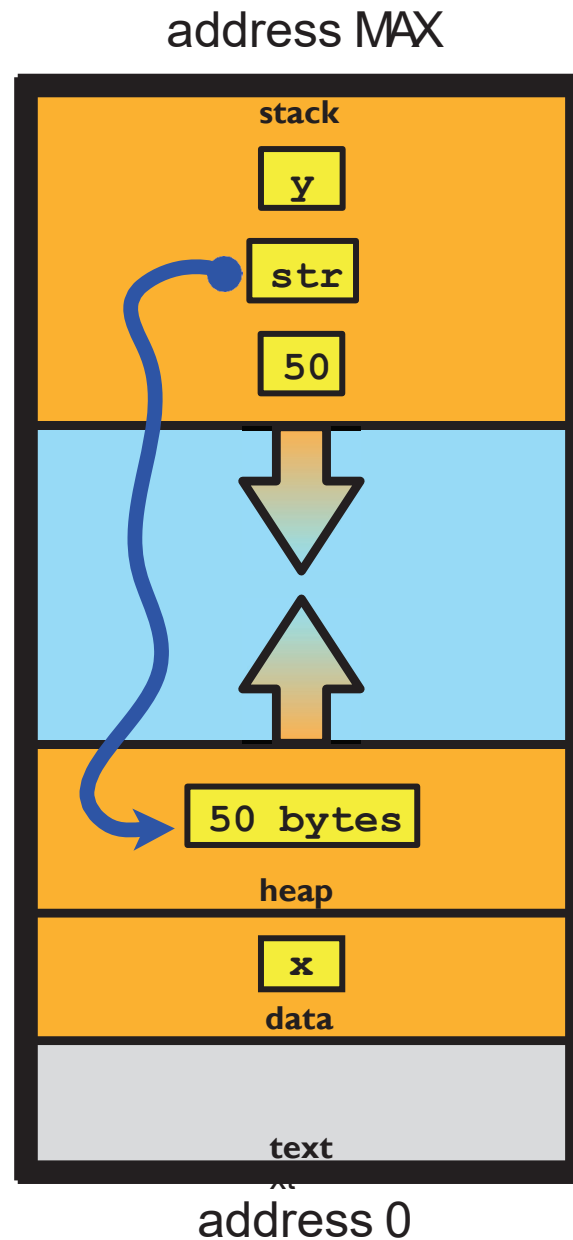
In general, the value of the **parameter** to the malloc might not be known at compile time. In this example the argument to malloc, the number 50 is pushed onto the **stack**.

Note: A compiler may also decide to put arguments in certain register.

```
// Example C program
#include <stdlib.h>

int x;

int main(void) {
    int y;
    char* str;
    str = malloc(50);
    exit(EXIT_SUCCESS);
}
```



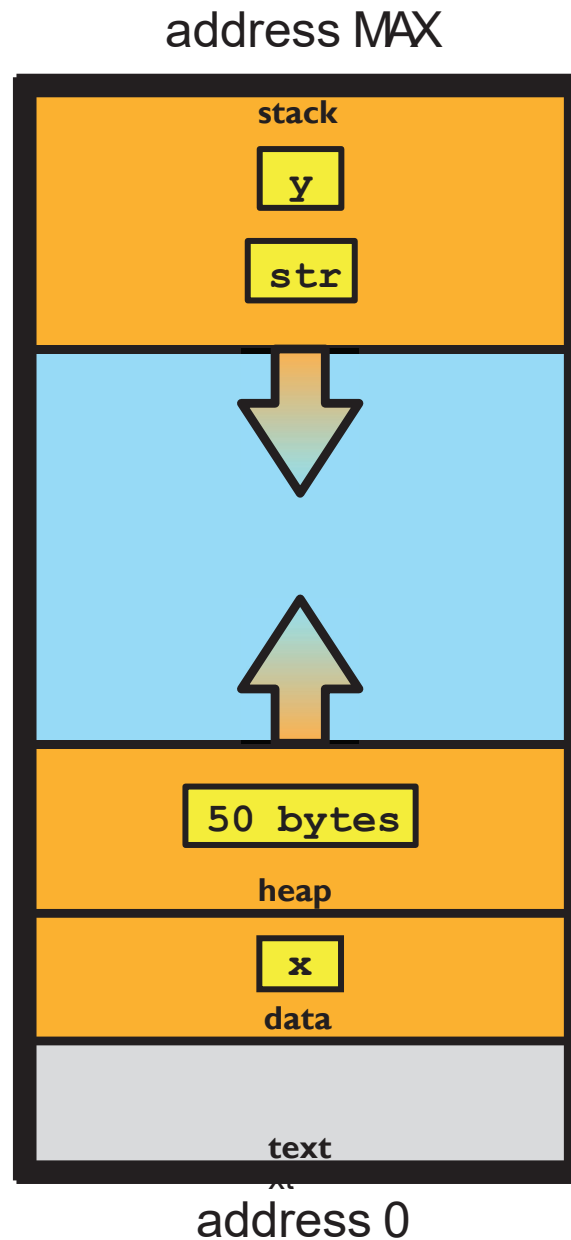
Now **malloc** has **dynamically** allocated memory on the heap.

The variable **str** stores the address to the first of the 50 bytes allocated on the heap, i.e., **str** is a **pointer**.

```
// Example C program
#include <stdlib.h>

int x;

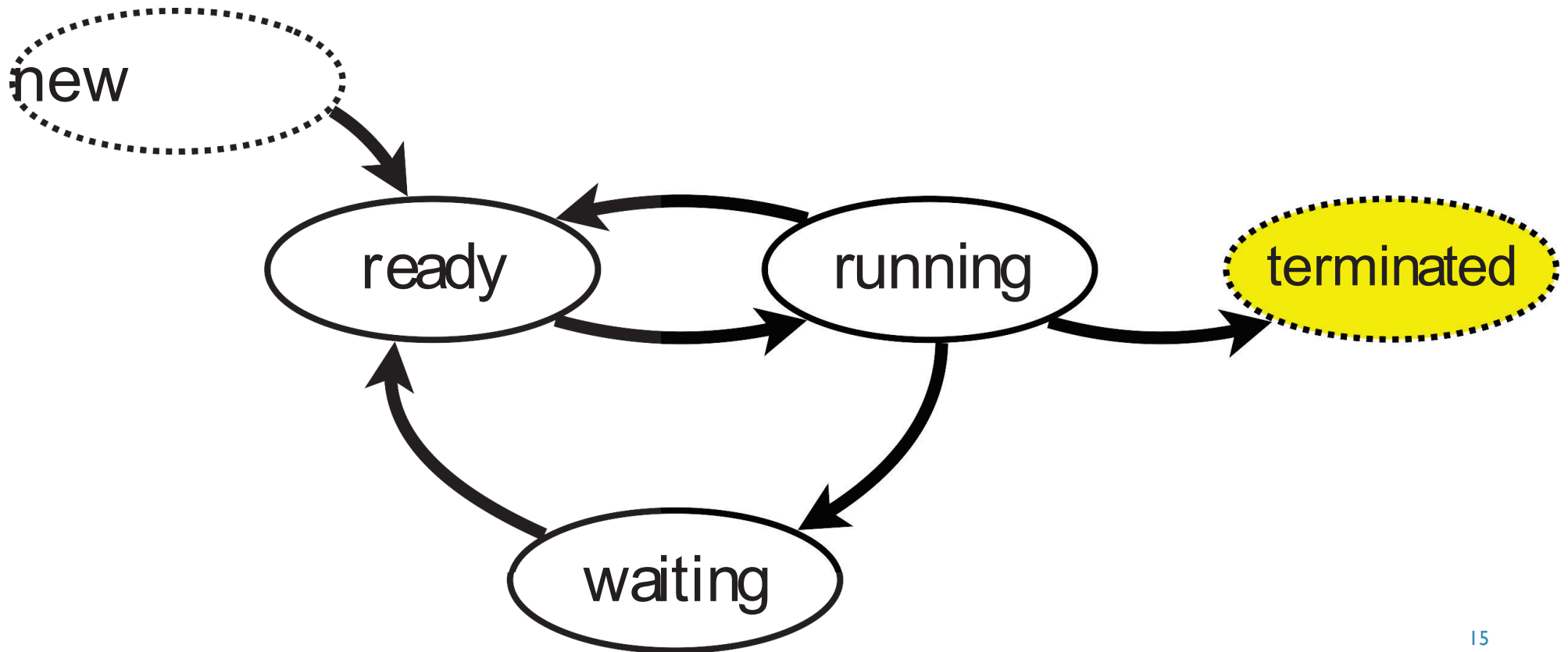
int main(void) {
    int y;
    char* str;
    str = malloc(50);
    exit(EXIT_SUCCESS);
}
```



Memory Deallocation

When `malloc` returns, the value 50, the argument used when calling `malloc` is no longer needed and is popped from the stack.

When the process terminates, the operating system can deallocate the memory used by the process.

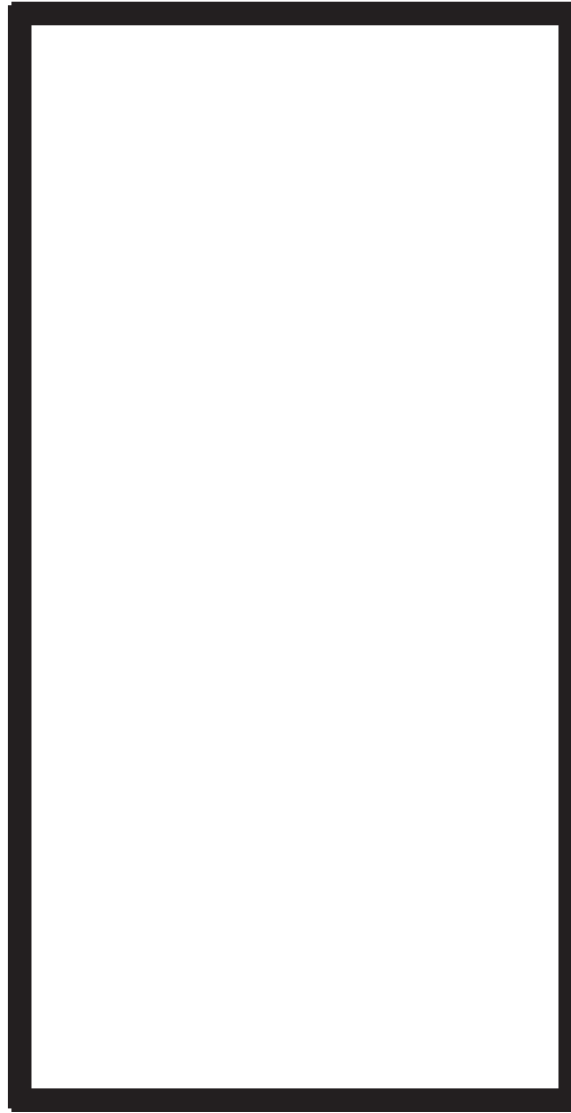


```
// Example C program
#include <stdlib.h>

int x;

int main(void) {
    int y;
    char* str;
    str = malloc(50);
    exit(EXIT_SUCCESS);
}
```

address MAX



address 0

Deallocated memory

The program terminates with a call to **exit()** and the memory allocated to the process can be deallocated.

Creating Processes

- In any operating system that is based on UNIX, processes all come from a single process : *init*
- When you start up the program, there is an initial process called *init* , which creates all other processes.
- So, basically, all processes running on the OS are really just **clones** of that initial process *init*.
- So, how do we create those clones; ie. new processes? Using **fork()**
- When we call **fork()**, we are just requesting a *clone* to be made of our process.

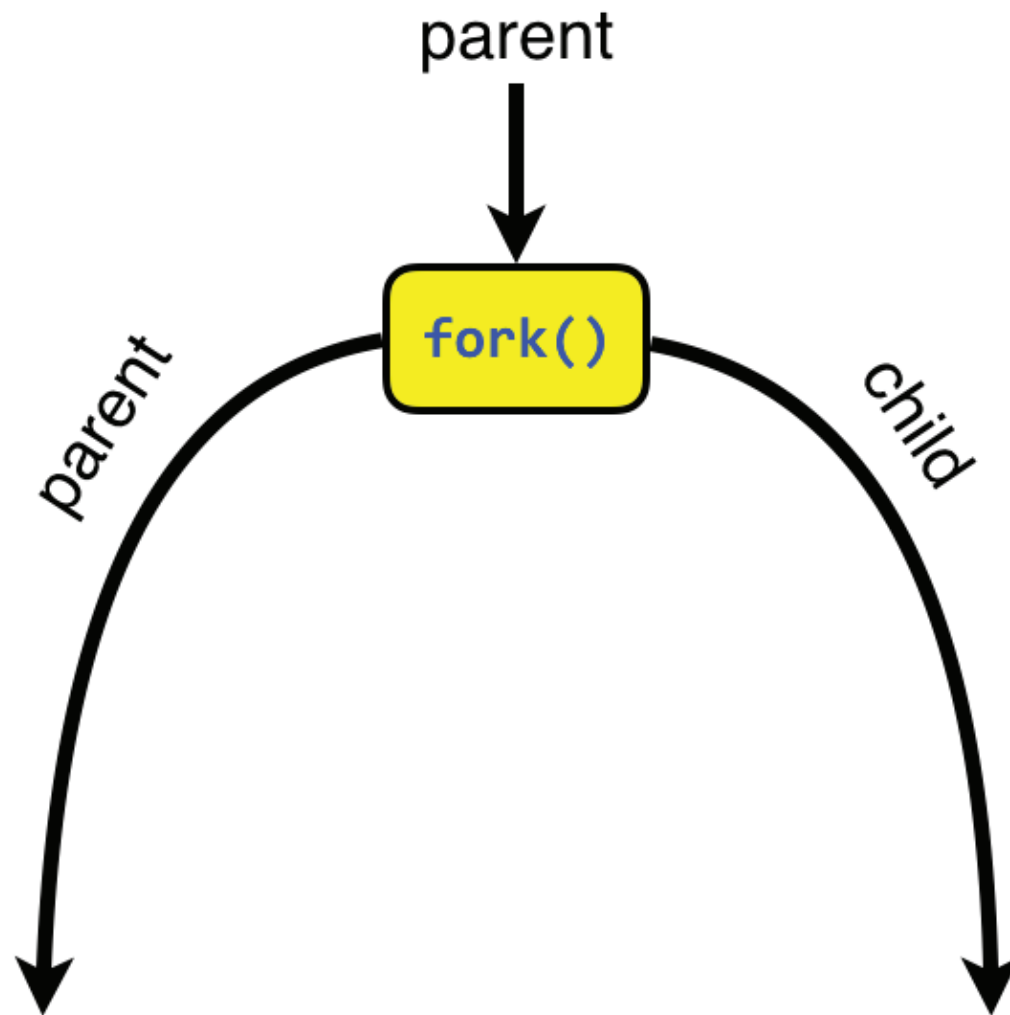
fork()

In Unix-like operating system, fork is an operation whereby a process **creates a copy** of itself.

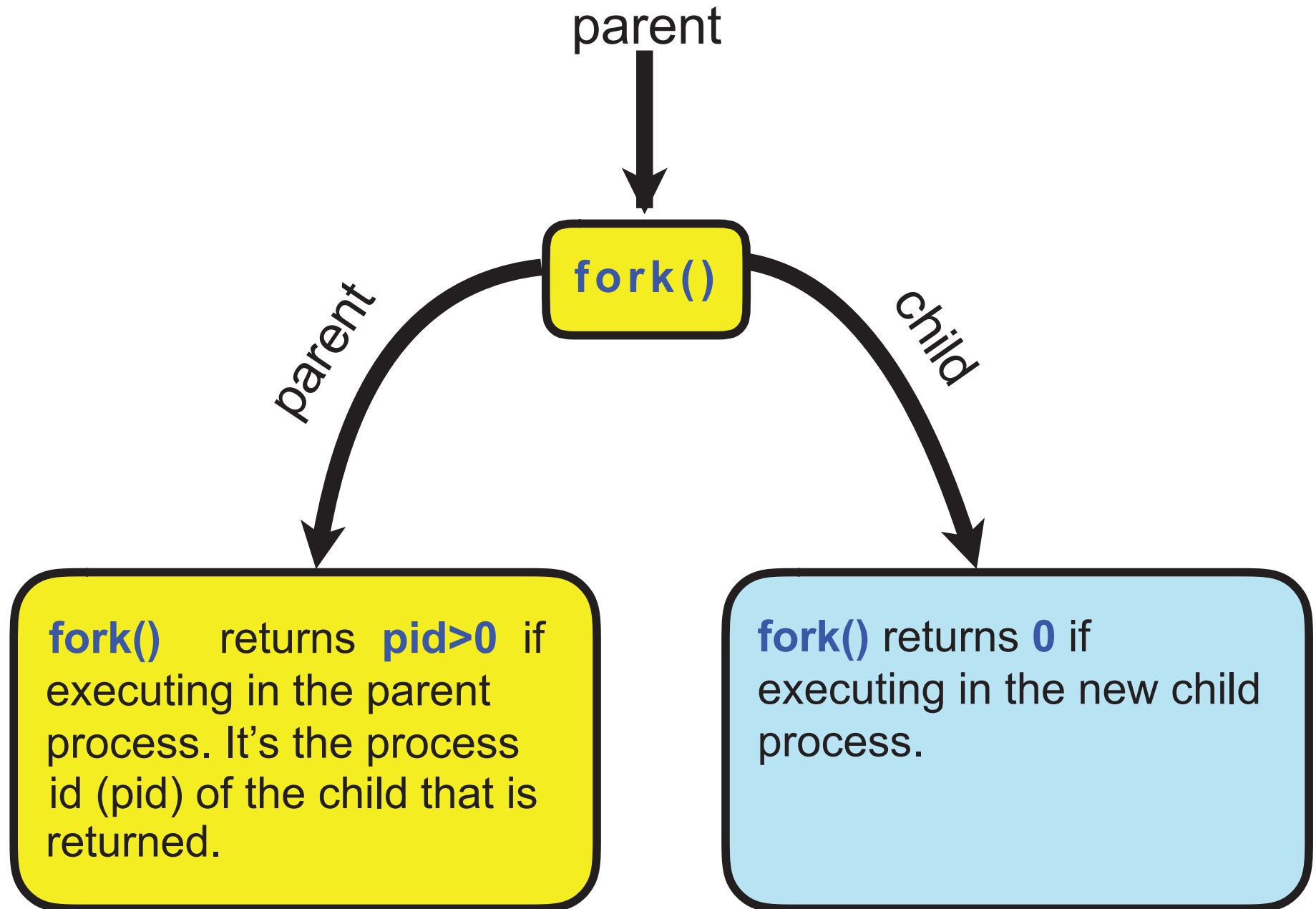


- The process calling fork() is called the **parent**.
- The new process is called the **child** of the parent.
- Fork is usually a **system call**, implemented in the kernel.

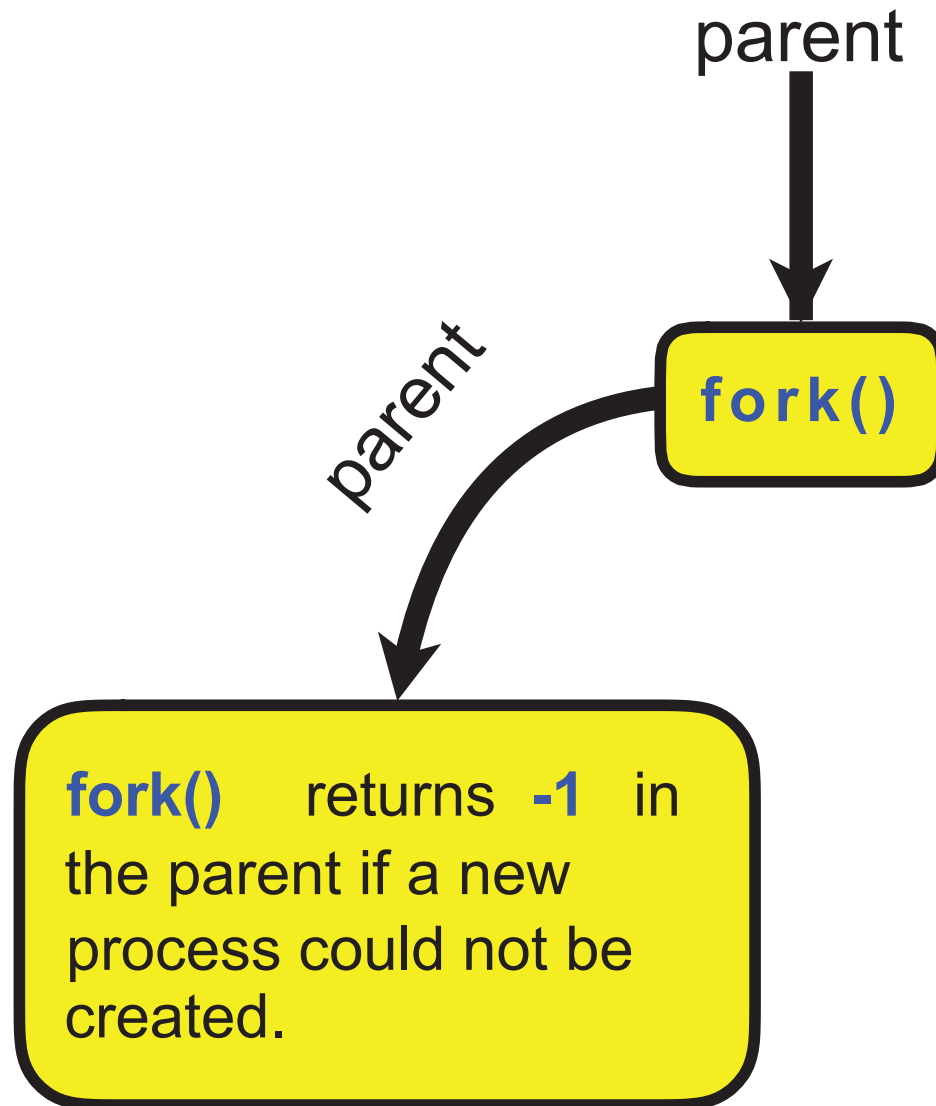
- The process invoking fork is called the **parent**.
- The new process created as the result of a fork is the **child** of the parent.



On success, **fork()** returns twice!



On failure, **fork()** returns **-1** in the parent

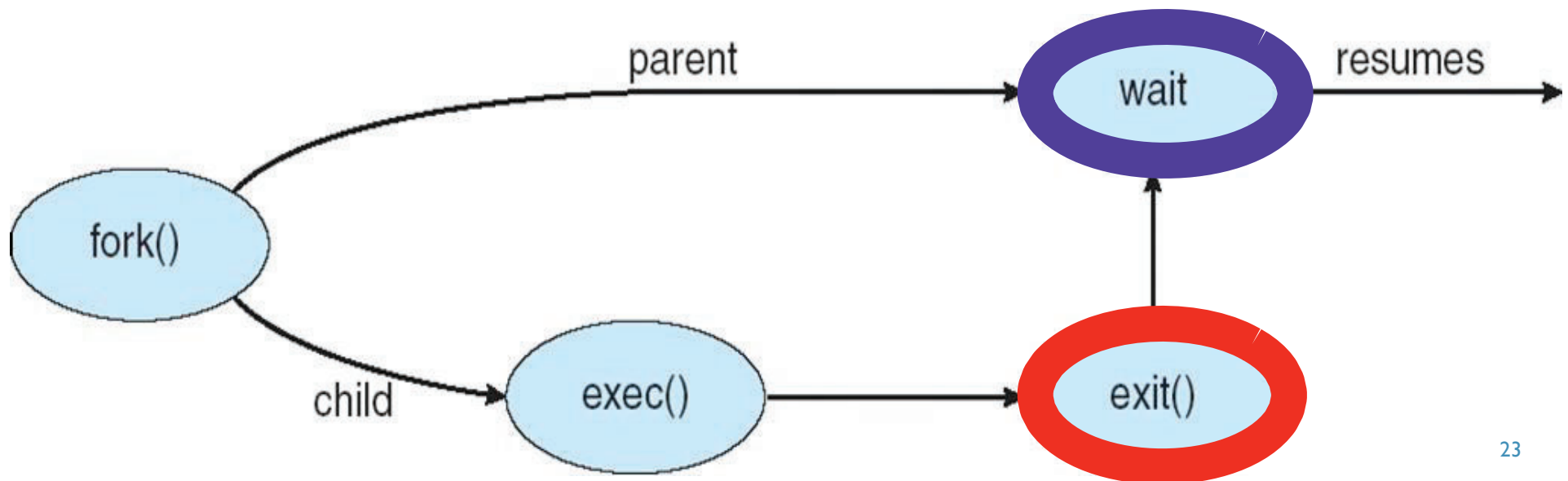


Orphan Processes

- An orphan process is a process whose parent process has **terminated**, though it remains running itself.
- Any orphaned process will be immediately adopted by the special **init** system process with **PID 1**.
- Processes execute **concurrently**.
- Both the parent process and the child process compete for the CPU with all other processes in the system.
- The operating systems decides which process to execute when and for how long.

wait()

- A call to `wait()` blocks the calling process until one of its child processes exits or a signal is received.
- After child process terminates, parent ***continues*** its execution after wait system call instruction.



wait()

- If the caller doesn't have any child processes, wait returns immediately without blocking the caller.
- Using `wait()`, the parent can obtain the exit status of the terminated child.
- If only one child process is terminated, then return a `wait()` returns process ID of the terminated child process.
- If more than one child processes are terminated, then `wait()` will return the process ID of an ***arbitrarily child process*** and then go on. (It won't wait for all of the child processes).

wait()

```
pid_t wait(int *status);
```

Status

- If status is not NULL, wait stores the **exit status** of the terminated child in the *int* to which status **points**.
- This integer can be inspected using the **WIFEXITED** and **WEXITSTATUS**.

Return value

- On **success**, wait returns the **PID** of the terminated child.
- On **failure** (no child), wait returns **-1**.



Zombie

A terminated process is said to be a zombie or defunct until the parent does **wait()** on the child.

- A zombie process is a process whose execution is completed but it still has an entry in the process table.
- Zombie processes usually occur for child processes, as the parent process still needs to read its child's exit status.
- Once this is done using the `wait()` system call, the zombie process is eliminated from the process table. This is known as **reaping** the zombie process.

Points related to zombie processes:

- All the memory and resources allocated to a process are deallocated when the process terminates using the `exit()` system call. But the process's entry in the process table is still available. This process is now a zombie process.
- The exit status of the zombie process can be read by the parent process using the `wait()` system call. After that, the zombie process is removed from the system. Then the process ID and the process table entry of the zombie process can be reused.
- If the parent process does not use the `wait()` system call, the zombie process is left in the process table. This creates a resource leak.
- If the parent process is not running anymore, then the presence of a zombie process indicates an operating system bug. This may not be a serious problem if there are a few zombie processes but under heavier loads, this can create issues for the system such as running out of process table entries.

Zombie vs Orphan Processes

Orphan

A child process that remains running even after its parent process is terminated or completed without waiting for the child process execution is called an orphan (parent did not invoke `wait()` and instead terminated). Then, it gets adopted by `init`.

Eg. An orphaned process might be created when its parent process crashes or terminates

A **Zombie** is a process that has completed its task but still, it shows an entry in the process table.